

Synthesizing niigo-lake through OpenROAD + ASAP7 — Scoping Report

Target: take `niigo_soc` (the RV64G OoO SoC, `src/niigo_soc.sv`, built `-DFPGA_BUILD`) from its current Xilinx-Vivado FPGA flow to an open-source RTL → GDSII ASIC flow on the **ASAP7 7.5-track 7 nm predictive PDK** driven by **OpenROAD** (with its integrated `syn` = yosys-slang elaboration + ABC mapping).

Every claim below was grounded in the RTL / PDK / OpenROAD source and adversarially verified by a second pass. File:line citations are to this checkout.

0. TL;DR verdict

Feasible, and easier than a typical FPGA → ASIC port — but not turn-key. The RTL is unusually portable: **zero instantiated vendor primitives** (no BUFG/MMCM/PLL/IBUF/xpm/ RAMB/DSP — only one `(* max_fanout *)` attribute), a single clock domain, no latches, no tri-states, uniform async-active-low reset, and a SystemVerilog style that yosys-slang (a full SV-2017 frontend) should elaborate as-is. The real work is in three buckets:

- 1. **Memory mapping (the dominant effort).** Everything that *infers* BRAM/LUTRAM on FPGA has no inferred-RAM path in an ASIC flow. ~715 Kbit of state, ~80 % concentrated in two structures (ITTAGE indirect-target tables 306 Kbit + L1I/L1D data 262 Kbit). A first GDS can flop everything (functionally correct, area-heavy); a real run maps the cache arrays to ASAP7 SRAM macros and shrinks/relocates the predictor tables.
- 2. **A handful of small, low-risk RTL edits** (reset synchronizer at the boundary; gate one sim-only fuzz block; restore `default_nettype`; trim the slang file list). No microarchitectural change required for a Tier-1 bring-up.
- 3. **Flow/platform assembly.** We have OpenROAD + the raw ASAP7 PDK, **not** the ORFS `platforms/asap7` glue (setRC, tracks, pdn, dont_use, KLayout layermap). Recommend lifting that platform from OpenROAD-flow-scripts rather than hand-authoring it.

SV-frontend readiness: the SystemVerilog is portable, but the integrated synth needed help. Running the real `sv_elaborate` (not just auditing) showed it elaborates *once* you (i) pass `--single-unit` and `--allow-use-before-declare`, (ii) define the string macro `LAB_18447` in a file rather than `+define+`, and (iii) resolve the **power operator** `**`, which OpenROAD's integrated `syn` backend does not implement (used by `memory_segments.vh` and pervasively by CVFPU). See §10 for the full validated findings and the two fixes (backend patch vs. external Yosys). No *RTL* construct is a hard blocker — the gaps are in the young integrated-synth backend and in flow flags.

1. Toolchain setup status

Component	State	Notes
ASAP7 PDK	✓ in place	<code>/home/mizuki/Desktop/workspace/asap7</code> — tech LEF (1x+4x), R/L/SL/SRAM cell LEF, GDS, CDL, and the <code>asap7_sram_0p0</code> fakeram macros (LIB/LEF/GDS/Verilog).

Component	State	Notes
ASAP7 TT Liberty	✓ extracted	The 20 TT-corner <code>.lib</code> were shipped as <code>.7z</code> ; extracted to <code>fpga/openroad/lib_tt/</code> (176 MB). The 5 <code>*_RVT_TT_*</code> are all that an RVT bring-up needs.
OpenROAD	🔧 building	Source checkout <code>26Q2-2457</code> ; integrated syn present (SV via yosys-slang). Being built from source (<code>DependencyInstaller -base/-common</code> → <code>Build.sh</code>) → <code>build/bin/openroad</code> .
ORFS asap7 platform	✗ not present	Only the OpenROAD <i>app</i> + raw PDK are here. See §6.

Setup gotcha already resolved: the dependency installer's `apt-get update` failed wholesale because of three broken third-party apt repos (`docker.list` → `ubuntu trixie`; google-chrome SSL resets through the `198.18.0.x` proxy; a malformed `forky-security` line in `sources.list`). The build driver temporarily neutralizes those (with backups) and **auto-restores** the original apt config. Network to github/gitlab/boost/debian is otherwise fine.

2. The flow (verified against this OpenROAD build)

OpenROAD now ships **integrated synthesis** (`src/syn`): `sv_elaborate <slang opts> → synthesize`. `sv_elaborate` runs the **slang** SV-2017 frontend; `synthesize` bit-blasts, runs ABC (`&fraig&dc2&if -K6`), maps sequential+combinationals to the read Liberty, and loads the gate netlist into ODB. **No separate Yosys/sv2v is required** for elaboration.

All flow commands the bring-up needs were confirmed present in *this* build (`define_cmd_args grep`): `read_lef/read_liberty/read_verilog/link_design`, `sv_elaborate/synthesize`, `initialize_floorplan/make_tracks/place_pins`, `tapcell/pdngen`, `global_placement`, `estimate_parasitics/set_wire_rc/set_layer_rc`, `buffer_ports/repair_design/repair_timing`, `clock_tree_synthesis`, `detailed_placement/filler_placement`, `global_route/detailed_route`, `write_def/write_db/write_verilog`, `place_macro/rtl_macro_placer`.

No Tcl write_gds — GDS stream-out is via the odb Python API (`odb.write_gds`) or the ORFS-standard external **KLayout** `def2stream.py` + layermap.

The 1×-vs-4× scale footgun (decided: use 1×)

ASAP7's drawn database is 4× true silicon; the PDK ships matched **1× (true, DATABASE MICRONS 1000, 1 nm grid)** and **4× (DATABASE MICRONS 4000)** LEF/GDS. OpenROAD's own asap7 regressions standardize on **1×** (39 test `.tcl` reference `*_1x_*`, `zero` reference `*_4x_*`). **Keep tech LEF + cell LEF + tracks + SRAM LEF + GDS stream-out all on 1×** or eat a 4× linear / **16× area** error. The std-cell LEF carries no **UNITS** — it inherits DBU from the **tech LEF read first**, so always `read_lef` the tech LEF before any cell/macro LEF.

Integrated-**syn** limitation that matters for macros

`src/syn/README.md`: `synthesize` **always flattens, loses instantiated-macro names, and cannot map latches**. Verified in source what "loses names" means: a `read_liberty` SRAM cell becomes a recognized **blackbox** (`elab/blackboxes.cc:219`) and survives as a real `dbInst` on its LEF master — but the **instance name is mangled to uNNNN** (`flow/export.cc:189`) and hierarchy is flattened. So integrated **syn** can carry

SRAM macros (select them by master type, not RTL name); if you need stable/named/hierarchical macro handling, run **external Yosys** for the macro flow and `read_verilog` the gate netlist into OpenROAD. niigo is fully flop-based, so the "no latch" limit is a non-issue.

3. RTL changes required

3a. Functional must-fix (1 item)

- **Boundary reset synchronizer.** The whole design is async-assert active-low `rst_l` with no synchronizer (`niigo_soc.sv`, `riscv_core_ooo.sv`). For ASIC add a 2-FF async-assert/sync-deassert synchronizer on `clk` at the `niigo_soc` boundary and feed the synchronized reset inward; SDC `set_false_path -from [get_ports rst_l]`. (The internal RTL is already uniformly async-low — only the boundary deassertion needs synchronizing.)

3b. Required build switch

- **Pass `-DSYNTHESIS` to slang.** It is load-bearing: it selects the plain-logic FP stub over a `real/$bitstoreal/$sqrt` behavioral model in `ooo_alu_pipe.sv:155-334`. Without it, `real` enters the cone and becomes a hard ABC blocker. (Every `fpga/synth/*.tcl` already defines it; the staged flow does too.) Build with the full FPGA define set: `-DRV64 -D000_4WIDE -DL1_CACHES -DL1D_CACHE -DAXI_MEMSYS -DFPGA_BUILD -DSYNTHESIS (+ LAB_18447="4b")`.

3c. Recommended hygiene (small, low-risk; not hard blockers)

Edit	File:line	Why
Gate the latency-fuzz block under <code>`ifndef SYNTHESIS</code> (force <code>fz_en=0</code>) <code>src/mem/niigo_memsys.sv:208-272</code> (<code>\$test\$plusargs/\$value\$plusargs/\$display initials</code>) The one ungated sim-only system-function leak in the synth cone. Vivado tolerates it (folds <code>plusargs</code> → <code>absent</code>); yosys-slang <i>should</i> too, but this is the single most likely thing to draw a warning/unsupported-task error. Match the codebase's own <code>ifndef SYNTHESIS</code> pattern (<code>uart.sv</code> , <code>ooo_alu_pipe.sv</code> , <code>active_list.sv</code>).		
Append <code>`default_nettype wire</code> <code>src/common_cells/lzc.sv</code> , <code>src/common_cells/rr_arb_tree.sv</code> Both open <code>none</code> and never restore. Vivado needs a <code>sed none→wire</code> shim (<code>run_soc.sh:25-27</code>); slang compiles per-unit so it's likely fine, but restoring the directive removes the need to port that shim and is the portable fix.		

Edit	File:line	Why
Drop CCD/CMI files from the slang file list	<code>cmi_*.sv, niigo_ccd_*.sv, niigo_dir*.sv, niigo_l1d_{moesi,gg}.sv</code>	Multicore (<code>CCD_AGENT</code>) files not instantiated by single-core <code>niigo_soc</code> . Vivado ignores read-but-unused modules; removing them avoids depending on slang's lazy-elaboration of uninstantiated parameterized modules. (The staged <code>gen_filelist.sh</code> already excludes them.)
Make <code>ecall_halt_en</code> a <code>localparam</code> /constant under SYNTHESIS	<code>src/ooo_alu_pipe.sv:38-41</code>	Cosmetic — replaces an <code>initial</code> -set power-on value with a constant; cleaner for ASIC.
(Optional) strip the two <code>(* max_fanout = 64 *)</code> attrs	<code>src/riscv_core_ooo.sv:443-444</code>	Vivado-only; inert in slang/OpenROAD. Replace with <code>set_max_fanout</code> in SDC + <code>repair_design</code> .

Sim-only audit result: 0 real blockers. Critically, there is **no `$readmemh/$readmemb/ $fopen` anywhere in the reachable set** — no memory array is file-initialized, so nothing zeroes/X-es an array on ASIC. All other `$display/$fatal/$error/assert/initial/wait/ final` in the cone are either gated (`ifndef SYNTHESIS / ifdef SIMULATION_18447 / ifdef AGENT_DEBUG`) or provably ignored/constant-folded by synth (procedural `$fatal` emits no hardware; `initial assert(constant)` never fires; CVFPU elaboration `$fatal/$warning` are `translate_off`/generate-guarded and inert for the niigo config).

3d. ASIC X-propagation watch-item (not a code change)

Under `SYNTHESIS` the branch predictors **drop their reset** (`tage_sc_l_predictor.sv:172-186, ittage_predictor.sv:125-131` wrap the reset in `ifndef SYNTHESIS`) so the tables infer BRAM on FPGA. On ASIC they become **unreset arrays that power up X**. Functionally safe (a predictor is a perf hint — bimodal counters self-train, tag misses predict not-taken), but gate-level sim will show X-prop until trained. Plan for `set_initial_condition`/forced-init in netlist sim, or accept it. (If mapped to SRAM later, the macro powers to a defined state.)

4. Memory mapping — the dominant effort

All state if **everything were flopped** $\approx 714,880$ bits (~ 715 Kbit / ~ 89 KB), independently recomputed and confirmed. Concentration is extreme — **two structures are ~ 80 %**:

Structure	Geometry (RV64, XLEN=64)	Bits	Port topology	ASIC recommendation
ITTAGE targets (ittage_predictor.sv)	$\text{base_target} + 3 \times \text{tage_target} = 4 \times (1024 \times 64)$	262,144	async-read, multi-bank	Shrink first. 64-bit targets in 1024-deep tables is extravagant — cut INDEX_BITS ($1024 \rightarrow 128/256$) and/or store PC-relative/low bits. To SRAM-ize needs registering the read (+1 cycle μ arch change).
L1I + L1D data (l1_data_array.sv)	$2 \times 4\text{-way} \times 64\text{-set} \times 512\text{b}$	262,144	sync-read 1R1W	SRAM macros (the prize). FSM-serialized R/W (see below) $\rightarrow$ address-merge refactor to single-port + way/width-banking.
ITTAGE rest (tags/conf/useful)	$3 \times 1024 \times 10 + 6 \times 1024 \times 2 + \text{valid}$	44,062	async	flop / bank-pack narrow banks
TAGE-SC-L tables	tags $3 \times 1024 \times 10 + 7 \times 1024 \times 2 + \text{sc_bias}$ 256×6	46,592	async	flop / bank-pack
L1I + L1D tag (l1_tag_array.sv)	$2 \times 4\text{-way} \times 64\text{-set} \times 52\text{b}$ (+L1I snoop replica)	39,936	L1D 1R1W; L1I 2R1W	L1D $\rightarrow$ SRAM; L1I tag is the only genuine 2-port case (live snoop) $\rightarrow$ keep flop/duplicate-tag or true 2-port macro. L1_TAG_BITS=52 is oversized vs Sv39 ($\sim 44\text{b}$) — trim.
LSQ entries_q	$16 \times \text{mem_entry_t}(1222)$	19,552	CAM	flops (fully-assoc)
ROB / active_list	$32 \times 347 + 4 \times 347 + 4 \times 710$	15,332	multiport	flops
Int IQ entries_q	$16 \times \text{issue_entry_t}(689)$	11,024	CAM wakeup	flops

Structure	Geometry (RV64, XLEN=64)	Bits	Port topology	ASIC recommendation
RAS <code>ras_stack_q</code>	128 × 64	8,192	1R1W (whole-array <code>_next</code> shadow)	restructure to a stack RAM, or leave flop
<code>phys_reg_file</code>	64 × 64, 18R / 4W	4,096	multiport regfile	flops (multiport RF, never a plain SRAM)
<code>fp_regs_q</code>	32 × 64, ~12R/4W	2,048	multiport	flops
TLB / free_list / busy_table / rename_map / branch_stack	—	~rest	CAM/multiport	flops

Key verified nuance (corrects the naive read)

The L1 **data** arrays *look* 1R1W (independent `ren/ridx` and `wen/widx` ports, so a local memory-inference pass infers a 2-port RAM that won't match the single-port `asap7_sram_0p0`), **but the cache FSMs gate read and write into mutually-exclusive states** — they never actually access the same cycle (`l1_icache.sv`: read in `S_SERVE/S_REPLAY`, write only in `S_INSTALL`; `l1_dcache.sv`: read in `S_IDLE/S_FLUSH_READ`, write only in `S_LOOKUP/S_INSTALL`). So they are **operationally single-port**: a mechanical *address-merge refactor* (`ren|wen→en`, `ridx|widx→addr`, add `we`) makes them map to the **1RW** fakeram. The genuine concurrent-R/W case is **only the L1I tag snoop port** (`tag_ren2 = snoop_valid` is async to the FSM and can coincide with a fetch read *and* an install write) — keep that as a flop/duplicate-tag filter or a true 2-port macro. Regardless of port refactor, the 512b line still needs **way-banking (×4)** and **width-banking** across multiple fixed-width fakeram words.

Two-tier strategy

- **Tier-1 — all-flop bring-up.** Every array legally synthesizes to flops (async-read and multiport are flop-friendly). ~715 Kbit of flops → large but functional GDS. **Before flopping, parameter-shrink the two monsters:** cut ITTAGE depth/target width, optionally reduce L1 sets/ways/line. This is the fastest route to a closed GDS and proves the flow.
- **Tier-2 — SRAM macros.** Map **L1I/L1D data + L1D tag** (262 K + 13 K, the clean wins) via the address-merge refactor + banking to `asap7_sram_0p0` 1RW macros. Leave the **L1I tag** (snoop) and all CAM/multiport/regfile state as flops. Converting the **ITTAGE/TAGE** tables to sync-read banked SRAM is a separate, latency-changing microarchitecture task — do it only if area demands, after the flow is proven.

(`asap7_sram_0p0` macro port, verified: single-port 1RW sync — `clk`, `ADDRESS`, `wd`, `banksel`, `read`, `write`, `dataout` + `sdel[4:0]` margin strap. e.g. `srmbank_128x4x32` = 512-deep × 32-wide.)

5. Top boundary & SDC

`niigo_soc` exposes `clk`, `rst_l`, `halted`, a **512-bit AXI4 master** `m_axi_*` (~1300 pins), and the FB1 debug taps `uart_*` + `dbg_probe`. Device space (CLINT/PLIC/UART) is inside the core, so only cacheable traffic + taps leave.

Recommended for a first GDS: keep `m_axi_*` as primary IO and constrain it with an SDC (the standard ASIC-block approach — no on-die DRAM, no new RTL). Drop/tie the FB1 debug plane (`uart_*`, `dbg_probe`

are observability-only, off any critical path) to shrink the boundary. A self-contained closed-netlist top (behavioral RAM/AXI-slave) is only worth it for full-chip gate-sim, as a *separate* sim wrapper — not the harden target (the design has zero tri-states, so a 1RW AXI slave is trivial if needed).

SDC skeleton (staged at `constraints.sdc`): one `create_clock` on `clk`;
`set_input_delay/set_output_delay` on all `m_axi_*`; `set_false_path -from [get_ports rst_1]`;
`set_max_fanout` to cover the `abort_mask` broadcast (the RTL attribute does not carry into OpenROAD).

6. Platform-assembly gaps vs ORFS

The raw PDK is LEF/GDS/LIB/CDL only — **none** of the OpenROAD flow glue. You must supply: the sorted/merged TT Liberty corner list; `set_layer_rc/set_wire_rc` (the tech LEF has geometry but *no per-layer R/C* → without it timing/repair is garbage); `make_tracks/track` offsets; the `pdn.cfg` grid recipe; the ASAP7 `dont_use` list (physical/fill/decap/tap/ tie cells, **all latches** `DHLx*/DLLx*` — integrated `syn` can't map latches anyway — clock-gates `ICGx*`, async-reset/scan flops); and the **KLayout layermap + def2stream scale** for GDS (cell GDS is the 4×-drawn artwork).

Real bring-up blocker found: the `asap7_sram_0p0` macro LEFs reference **SITE coreSite**, which is never **defined anywhere in the PDK** (and the tech LEF defines no SITE). So `initialize_floorplan -additional_sites coreSite` alone is insufficient — you must either define a `coreSite` SITE in a small supplemental LEF or `sed`-remap the SRAM LEFs' `SITE coreSite` → `SITE asap7sc7p5t`. Plan for this in any SRAM-macro run.

Recommendation: `git clone OpenROAD-flow-scripts` and lift `flow/platforms/asap7/` rather than hand-rolling — it already contains the validated `setRC.tcl`, `tracks`, `pdn.cfg`, canonical `dont_use`, tapcell distances + site wiring, and the KLayout layermap/scale that reconciles the 4× GDS. Point its paths at this PDK and drive the locally-built `openroad` binary. Hand-authoring all of that is exactly where people lose days to the scale footgun.

7. Phased plan

- **P0 — Setup (in progress).** Finish the OpenROAD build; extract ASAP7 (done); obtain the ORFS `platforms/asap7` glue (§6). Exit: `openroad -version` runs; platform RC/tracks/pdn/`dont_use`/layermap on hand.
- **P1 — Elaborate + synth sanity (no PDK needed for the first half).** Apply §3 edits (reset sync optional here; gate fuzz block; default_nettype; trim file list). Run `sv_elaborate` + `synthesize` against the 5 RVT TT libs; confirm the SV frontend clears and the gate netlist builds. Exit: clean elaboration + a mapped netlist + cell/area report.
- **P2 — Tier-1 flop-mapped GDS.** Add the boundary reset synchronizer + SDC; **param-shrink ITTAGE** (and optionally L1) to keep flop area sane; run floorplan → place → CTS → route with the ORFS asap7 platform values. Exit: a routed `niigo_soc` DEF/GDS at a relaxed clock, DRC/LVS-clean.
- **P3 — Tier-2 SRAM macros.** Address-merge refactor on `l1_data_array/L1D l1_tag_array`; bank to `asap7_sram_0p0`; resolve `coreSite`; macro-place + PDN + route. Exit: cache data/tag in SRAM, materially smaller area, better timing.
- **P4 — Full RV64 + push timing.** Restore full ITTAGE if desired; optional real ICG for CVFPU clock-gating; iterate `repair_timing/floorplan` toward a target period.

(Optional accelerator for P1/P2: a first bring-up in **RV32** (`unset RV64`, FP32) halves the datapath/CVFPU and de-risks the flow before scaling to RV64+RV64D.)

8. Risk register

Risk	Likelihood	Impact	Mitigation
OpenROAD source build fails (gcc 15.2 bleeding-edge / dep pins)	Med	High (no tool)	Default build isn't -Werror ; fallbacks = Precision-Innovations prebuilt binary, Docker image, or Bazel build.
1×/4× scale mix-up	Med	High (16× area)	Lock everything to 1×; lift ORFS scale/layermap; never mix tech+cell scales.
coreSite undefined breaks SRAM floorplan	High (if Tier-2)	Med	Define/remap site before floorplan (§6).
yosys-slang version quirk on a construct	Low	Med	Frontend audited clean; gate fuzz block; trim file list; Vivado FB2 already elaborates the identical tree.
Flop area at full config too large to place	Med	Med	Tier-1 param-shrink (ITTAGE first); Tier-2 SRAM; RV32 first.
ABC runtime/scale on full OoO+CVFPU netlist	Low–Med	Med	Synthesize incrementally; -reduce_name_loss off for QoR; shrink predictors.

9. Staged flow artifacts (in **fpga/openroad/**)

Ready-to-run, **pending the OpenROAD binary** — grounded in verified command names/paths but not yet executed end-to-end. Treat as a starting harness, not a validated flow.

- **lib_tt/** — extracted ASAP7 TT-corner Liberty (done).
- **gen_filelist.sh** — emits a slang command-file (**design.f**): include dirs + the full **-DRV64 ... SYNTHESIS** define set + the CVFPU source order + **src/*.sv + src/mem/*.sv** minus the sim/alt **and** CCD files.
- **dont_use.tcl** — ASAP7 **set_dont_use** globs (latches/fill/decap/tap/tie/scan/ICG).
- **constraints.sdc** — clock + AXI IO delays + reset false-path + max_fanout skeleton.
- **flow_asap7.tcl** — Tier-1 all-standard-cell flow (read libs/LEF 1× RVT → elaborate → synthesize → floorplan → place → CTS → route → write DEF/DB/V).
- **run.sh** — driver: regenerates the filelist and invokes **openroad flow_asap7.tcl**.
- **README.md** — quick orientation + current setup state.

10. Validation findings (from actually running **sv_elaborate**, not just auditing)

The static audit predicted "SV-frontend READY." Running the **real** OpenROAD **sv_elaborate** on the full **niigo_soc** tree surfaced four concrete issues the audit missed — three are flow flags, one is an OpenROAD backend gap. With all four addressed, the frontend elaborates.

1. **--single-unit required**. Every **.sv** **`includes** the package-defining **.vh** (**000_Types**, **RISCV_ISA**, **Internal_Defines**, **NIIGO_Mem**, **RISCV_Priv**, ...). slang defaults to *per-file* compilation units, so each file got its own copy → **error: duplicate definition of '000_Types'** etc. The design relies on

single-unit compilation (as Verilator/Vivado do when handed all files together). Fix: pass `--single-unit`. (This also makes the `$unit`-scope typedefs in `superscalar_types.vh` and the `default_nettype` discipline behave — so still append ``default_nettype wire` to the two `common_cells`.)

2. **--allow-use-before-declare required.** slang enforces declare-before-use; the design has forward references Vivado/Verilator tolerate: `load_store_queue.sv:90` uses `ADDR_SHIFT` in a port width (localparam declared at `:131`); `niigo_memsys.sv:312` uses `present_dmem/l1d_req_fire` in an `assign` above their declarations (`:539/:550`). Fix: pass `--allow-use-before-declare` (or reorder the declarations in RTL — the flag is the zero-touch option).

3. **String-valued macro `LAB_18447` must be defined in a file, not on the command line.**

`riscv_uarch.vh` computes `MEMORY_READ_WIDTH/*_READ_DELAY` via `(`LAB_18447 == "4a") ? ...`. A slang command-file `+define+LAB_18447="4b"` arrives `empty ((== "4a") → parse error)` — the valueless `+define+s` are fine, only the string value is dropped. Fix: emit ``define LAB_18447 "4b"` into a leading source file (under `--single-unit` it is visible design-wide). `gen_filelist.sh` does this.

4. **OpenROAD integrated `syn` doesn't implement the power operator `**`** — the real blocker. After 1–3, elaboration's AST stage passes ("Top level design units: `niigo_soc`"), then the **synthesis backend aborts**: `slang_frontend::BackendGraphBuilder::Biop → log_error("Unsupported binary operator")` (`src/syn/src/elab/backend_builder.cc:595`). Its `Biop` switch handles `+ - * / % & | ^ ~^ == != < <= > >= && || << >>` but **not `**`** (nor `==?/!=?/->/<->`). `niigo` hits it at `memory_segments.vh:44` (`512 * 2 ** 10`) and **CVFPU uses `**` pervasively** (`2**EXP_BITS`, `2** (MAN_BITS-1)`), biases — `fpnew_fma.sv`, `fpnew_cast_multi.sv`, `fpnew_pkg.sv`, ...). Two resolutions:

- **(a) Patch the backend** — add `Power` cases to `backend_builder.cc::Biop`. **Done in this workspace's OpenROAD checkout**, in BOTH places (CVFPU needs both): a *constant-fold* case in the const path (`base**exp` by repeated `Const::mul`) for `2**10/2**EXP_BITS`, and a *graph-path* case for a runtime exponent with a constant power-of-two base (`2^k ** y → Shl(1, y, k)`) for CVFPU's multi-format `2**exp_bits(fmt_signal)`. After both patches the **"Unsupported binary operator" abort is gone** — elaboration proceeds past every operator (confirmed by re-running). *This is a local modification to the vendored OpenROAD tree* (`src/syn/src/elab/backend_builder.cc`) — track it if you update OpenROAD, or upstream it.
- **Measured outcome (the definitive result):** with the 3 flags + both `**` patches, the slang **frontend elaborates the full RV64 OoO+CVFPU `niigo_soc` cleanly** — **Build succeeded: 0 errors, 0 warnings** ("Top level design units: `niigo_soc`"). **Correctness is validated.** The remaining wall is purely **backend scale/performance**: `sv_elaborate`'s word-level netlist build (`populate_netlist`, which flattens everything) did **not finish within a 30-minute cap** (100% CPU throughout, RSS growing slowly 6 → 8.7 GB — no OOM, no error). The integrated `syn` is young and does not scale comfortably to a design this large. For a practical integrated-`syn` bring-up, **reduce config first** (RV32 instead of RV64, and/or drop the heavy CVFPU multi-format FPU) to shrink the flattened netlist, then scale up — or use route (b), external Yosys.
- **(b) Use external Yosys** (the full `yosys-slang` plugin) for synthesis, then `read_verilog` the gate netlist into OpenROAD. Upstream `yosys-slang` supports `**`. This is the more robust production path and also gives named/hierarchical macro control that integrated `syn` discards (§2) — recommended if the backend patch proves insufficient for the runtime-`**` cases.

Net: the SV is portable, but OpenROAD's *integrated* synthesis is young — expect to either carry small backend patches (`**`, and possibly more operators as coverage grows) or drive synthesis with external Yosys. The four

fixes above are captured in the staged flow scripts (`gen_filelist.sh` defines `LAB_18447`;
`flow_asap7.tcl/elaborate_check.tcl` pass `--single-unit --allow-use-before-declare`).